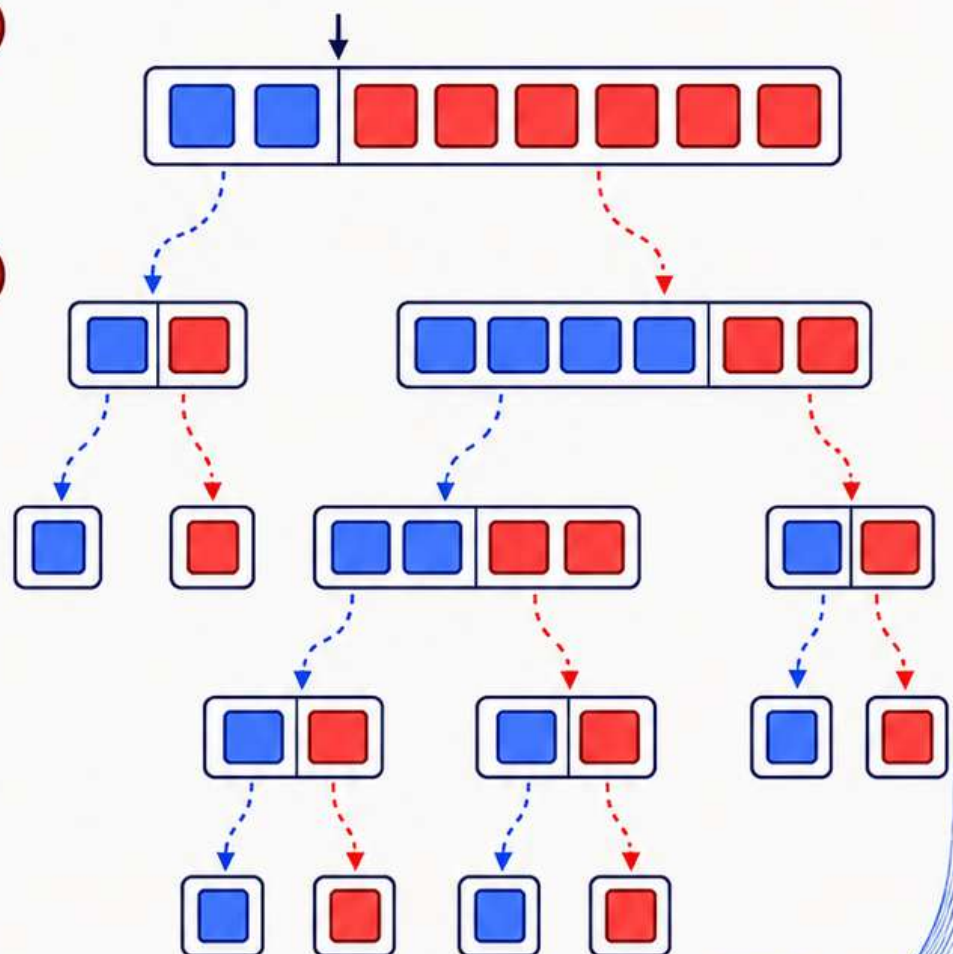
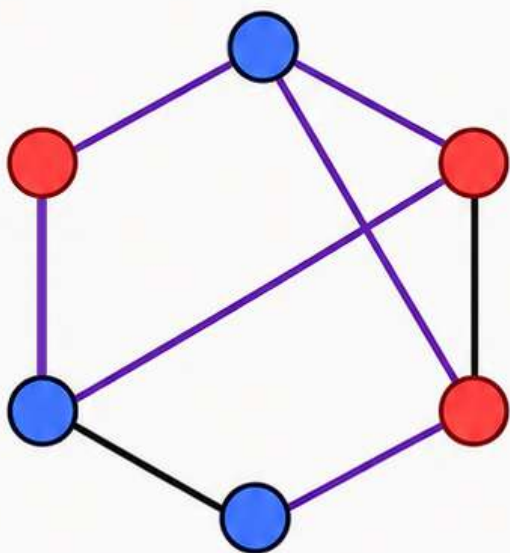


Randomised Algorithms

Sahil M Deo



Contents

1	Introduction	5
1.1	What is a Randomised Algorithm?	5
1.2	Implementation of Randomness in C++	5
1.3	Problems	8
2	Algorithmic Performance	9
2.1	Worst Case Analysis	9
2.2	Average Case Analysis	9
2.3	Time Complexity Analysis	10
2.4	Problems	12
3	Complexity Classes	13
3.1	Decision Problems	13
3.2	P	13
3.3	NP	14
3.3.1	The Question of P versus NP	14
3.4	The Class $co-NP$	14
3.5	Polynomial-Time Reductions	14
3.6	NP -Complete Problems	15
3.7	Randomized Complexity Classes	15
3.7.1	The Class RP	16
3.7.2	The Class $co-RP$	16
3.7.3	The Class ZPP	16
3.7.4	The Class BPP	16
3.8	Probabilistic Polynomial-Time Algorithms	16
3.9	Relationships Between the Classes	17
4	The Classics	18
4.1	QuickSort	18
4.1.1	The Algorithm	18
4.1.2	Worst Case Runtime	18
4.1.3	Expected Runtime	19
4.2	QuickSelect	20
4.2.1	The Algorithm	20
4.2.2	Expected Runtime	21
4.2.3	Expected Recursion Depth	21
4.3	Problems	23

5	Hashing	24
5.1	What even is Hashing?	24
5.2	Polynomial Multiplication	24
5.3	Matrix Multiplication	25
5.3.1	Freivalds' Algorithm	25
5.4	A Matrix Problem	26
5.4.1	Deterministic Method	27
5.4.2	Hashing Method	27
5.5	A Sliding Window Problem	29
5.6	Problems	31
6	Data Structures	32
6.1	Skip Lists	32
6.1.1	The Structure	34
6.1.2	Expected Height	34
6.1.3	Search	35
6.1.4	Insertion	37
6.2	Treaps	38
6.2.1	Binary Trees	38
6.2.2	Tree Traversal	39
6.2.3	Binary Search Trees	39
7	Number-Theoretic Algorithms	59
7.1	Introduction	59
7.2	Fermat Test	59
7.2.1	Fermat's Little Theorem	60
7.2.2	The Algorithm	60
7.2.3	Error Probability	60
7.3	Miller–Rabin Test	61
7.3.1	A Key Property	61
7.3.2	The Algorithm	62
7.3.3	Error Probability	62
7.3.4	Performance	63
7.4	Pollard's Rho Algorithm	63
7.4.1	Floyd's Cycle-Detection Algorithm	64
7.4.2	The Algorithm	66
7.4.3	Performance	67
7.5	Problems	70
8	Probabilistic Method	71
8.1	Dominating Set	71
8.2	Bipartite Subgraph	73
8.2.1	Expected Number of Bichromatic Edges	73
8.2.2	A Simple Randomised Algorithm	74
8.2.3	Derandomisation	74
8.3	Array Construction	76
9	Online Algorithms	79
9.1	Dating	79
9.2	Load Balancing	82
9.3	Distinct Elements	84
9.3.1	Flajolet–Martin Method	84

9.3.2	HyperLogLog Method	86
10	Heuristic Algorithms	87
10.1	Dominating Set Revisited	87
10.2	Independent Set	89
10.2.1	Random Permutation	89
10.2.2	Local Decision	90
10.3	Bipartite Subgraph Revisited	92
10.3.1	Self-Correction	92
	Appendices	93
A	Probability and Expectation	94
A.1	Linearity of Expectation	94
A.2	Tail Sum Formula	94
A.3	Union Bound	94
A.4	Independence of Expectation	94
A.5	Variance	94
A.6	Indicator Variables	95
A.7	Law of Total Expectation	95
A.8	Wald's Equation	95
B	Inequalities	97
B.1	Exponential Inequality	97
B.2	Jensen's Inequality	97
B.3	Markov's Inequality	98
C	Convergence and Limit Theorems	99
C.1	Identical and Independently Distributed Random Variables	99
C.2	Almost Surely	99
C.3	Almost Never	99
C.4	$\lim_{n \rightarrow \infty} X_n = Y$	100
C.5	Convergence in Probability	100
C.6	The Weak Law of Large Numbers	100
C.7	Almost Sure Convergence	101
C.8	A_n infinitely often	101
C.9	The Borel–Cantelli Lemmas	101
C.10	The Strong Law of Large Numbers	102
C.11	Convergence in Distribution	102
C.12	The Central Limit Theorem	103
D	Discrete Mathematics	104
D.1	Pigeonhole Principle	104
D.2	Graph Theory	104
E	Linear Algebra	106
E.1	Vector Spaces	106
E.2	Subspaces	106
E.3	Dimension of a Subspace	106
E.4	The Nullspace	107
E.5	Nullity and Rank	107
E.6	The Rank–Nullity Theorem	107

Chapter 3

Complexity Classes

Given what we have already seen, what is the best time complexity for a problem? When can we hope to come up with a *fast* algorithm?

This is exactly what complexity classes are about. While the classes themselves are defined rigorously, there are some very important things which we have not yet proved – they remain open problems and we believe them to be true in practice...

3.1 Decision Problems

Complexity classes are traditionally defined using *decision problems*. A decision problem is a computational problem whose answer is either YES or NO.

Examples

- Given a graph G , is G connected?
- Given an integer m , is m prime?
- Given a graph G and an integer k , does G contain a clique¹ of size at least k ?

Although not all computational tasks are naturally decision problems, most can be reformulated as one.

3.2 P

The class P consists of all decision problems that can be solved by a deterministic algorithm in at most polynomial time. Let Q be a decision problem and $T(Q)$ be the best time complexity² of a deterministic algorithm for Q . Then $T(Q) \leq \Theta(n^c)$ for some constant $c \geq 1$.

$$P = \bigcup_{c \geq 1} \{Q : T(Q) \leq \Theta(n^c)\}$$

Decision Problem(Q)	Algorithm	Runtime ($T(Q)$)
Primality Test	AKS	$\Theta(m^{12})$
Graph Connectivity	BFS	$\Theta(V + E)$

¹Refer Appendix D.2

²This time complexity is in terms of the input length and not the input number itself – a number n when taken as input is represented by a bit string of size $m = \Theta(\log n)$. So an algorithm which takes $O(n)$ time will be considered as an exponential time algorithm since $\Theta(n) = \Theta(2^m)$. This is why the normal method of trying all numbers from 1 to $\sqrt{n}$ will not prove that primality testing of a number is in P .

3.3 NP

The class NP consists of all decision problems whose answer when "YES" can be verified in polynomial time. The algorithm which solved the problem outputs a *certificate* along with "YES" – this certificate is information which can be used to verify in polynomial time that the answer is indeed "YES".

Decision Problem (Q)	Certificate	Verification Time
Clique of size k	Set of k vertices	$\Theta(k^2)$
Graph Coloring of k colors	Vertex : Color mapping	$\Theta(V + E)$

An obvious conclusion from the definition of NP is that $P \subseteq NP$ because any problem that can be solved efficiently can also be verified efficiently.

3.3.1 The Question of P versus NP

Every problem that can be solved efficiently can certainly be verified efficiently. Therefore,

$$P \subseteq NP.$$

The central open problem in theoretical computer science is whether this inclusion is strict.

The P versus NP Problem. Is

$$P = NP; ?$$

No proof is currently known.

If $P = NP$, then every problem whose solutions can be verified efficiently can also be solved efficiently.

3.4 The Class $co-NP$

The class $co-NP$ contains complements of problems in NP .

Definition (Class $co-NP$). A decision problem belongs to $co-NP$ if its complement belongs to NP .

Equivalently, a NO answer admits a polynomially verifiable certificate.

For example, the complement of the Clique problem asks whether a graph contains no clique of size at least k .

We currently do not know whether

$$NP = co-NP.$$

3.5 Polynomial-Time Reductions

To compare the difficulty of computational problems, we use reductions.

Definition (Polynomial-Time Reduction). A problem A is polynomial-time reducible to a problem B , written

$$A \leq_p B,$$

if every instance of A can be transformed into an equivalent instance of B in polynomial time.

Intuitively, if $A \leq_p B$ and we know how to solve B efficiently, then we can also solve A efficiently.

Reductions allow us to transfer algorithmic techniques as well as hardness results between problems.

3.6 NP-Complete Problems

Some problems are at least as hard as every problem in NP .

Definition (NP-Complete). A problem L is NP-complete if

1. $L \in NP$,
2. For every problem $A \in NP$,

$$A \leq_p L.$$

Thus every problem in NP can be transformed into L using only polynomial time.

Theorem. If any NP-complete problem belongs to P , then

$$P = NP.$$

Examples of NP-Complete Problems.

- SAT
- 3-SAT
- Clique
- Vertex Cover
- Hamiltonian Cycle
- Subset Sum

The discovery of NP-completeness provides strong evidence that these problems are unlikely to possess polynomial-time algorithms.

3.7 Randomized Complexity Classes

We now introduce complexity classes that permit algorithms to use random bits during computation.

3.7.1 The Class RP

Definition (Randomized Polynomial Time). A decision problem belongs to RP if there exists a randomized polynomial-time algorithm satisfying:

- If the correct answer is **NO**, the algorithm always outputs **NO**.
- If the correct answer is **YES**, the algorithm outputs **YES** with probability at least $\frac{1}{2}$.

Thus an RP algorithm may make false negatives but never false positives.

Repeating the algorithm independently reduces the error probability exponentially.

3.7.2 The Class $co-RP$

Definition ($co-RP$). A problem belongs to $co-RP$ if there exists a randomized polynomial-time algorithm satisfying:

- If the correct answer is **YES**, the algorithm always outputs **YES**.
- If the correct answer is **NO**, the algorithm outputs **NO** with probability at least $\frac{1}{2}$.

Such algorithms may make false positives but never false negatives.

3.7.3 The Class ZPP

Definition (Zero-Error Probabilistic Polynomial Time). A problem belongs to ZPP if there exists a randomized algorithm that

- never outputs an incorrect answer,
- has expected polynomial running time.

An important characterization is

$$ZPP = RP \cap co-RP.$$

3.7.4 The Class BPP

Definition (Bounded-Error Probabilistic Polynomial Time). A decision problem belongs to BPP if there exists a randomized polynomial-time algorithm whose probability of producing the correct answer is at least $\frac{2}{3}$ on every input.

The constants $\frac{2}{3}$ and $\frac{1}{3}$ are arbitrary. By repeating the algorithm several times and taking a majority vote, the error probability can be reduced exponentially.

Thus BPP captures problems that can be solved efficiently with an arbitrarily small probability of error.

3.8 Probabilistic Polynomial-Time Algorithms

The term *Probabilistic Polynomial Time* (PPT) is frequently used in randomized algorithms and cryptography.

Definition (PPT Algorithm). A PPT algorithm is a randomized algorithm that

1. uses random bits during execution,
2. runs in polynomial time.

Unlike classes such as RP or BPP , the term PPT refers to a model of computation rather than a class of decision problems.

Examples.

- Randomized QuickSort.
- Miller–Rabin primality testing.
- Karger’s minimum cut algorithm.

All of these are PPT algorithms.

3.9 Relationships Between the Classes

The following inclusions are known:

$$P \subseteq ZPP \subseteq RP \subseteq BPP.$$

Similarly,

$$P \subseteq ZPP \subseteq co\text{-}RP \subseteq BPP.$$

Moreover,

$$ZPP = RP \cap co\text{-}RP.$$

We also know that

$$P \subseteq NP, \quad P \subseteq co\text{-}NP.$$

Many other relationships remain unknown. In particular, we do not know whether

$$P = NP, \quad NP = co\text{-}NP, \quad BPP = P.$$

These questions lie at the heart of modern complexity theory.

Summary

- P contains efficiently solvable decision problems.
- NP contains problems whose **YES** instances can be verified efficiently.
- $co\text{-}NP$ contains problems whose **NO** instances can be verified efficiently.
- NP-complete problems are the hardest known problems in NP .
- RP , $co\text{-}RP$, ZPP , and BPP model different forms of randomized computation.
- A PPT algorithm is simply a randomized polynomial-time algorithm.
- The relationship between many of these classes remains unknown.